

Software Engineering - 5CM505

Software Deployment and Maintenance

Ioannis Tsioulis (MC)



*slides acknowledgements
Wajahat Ali Khan (UoD)*

Content

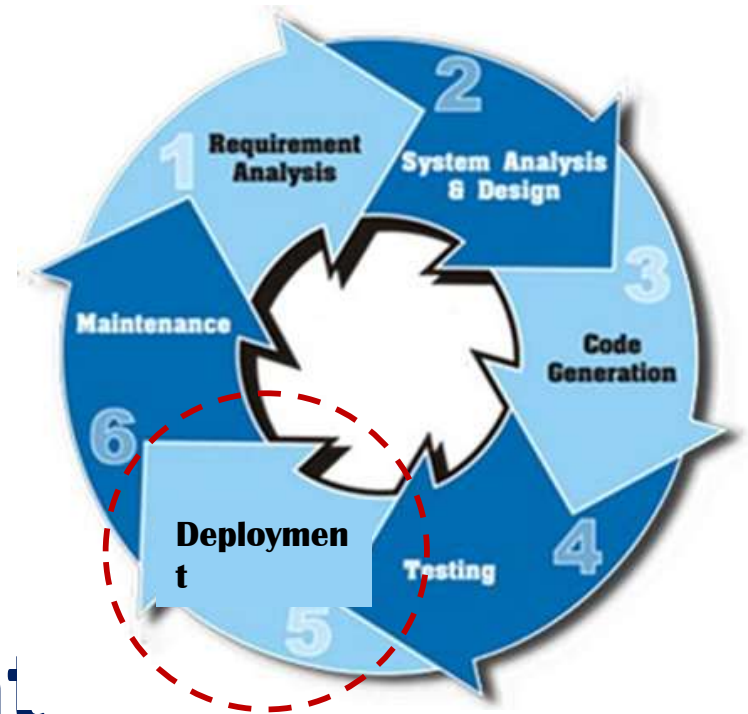
- Software Deployment
 - Considerations
 - Case Study
 - Deployment Models
 - Big Bang
 - Phased
 - Parallel
- Software Maintenance
 - Types
 - Costs



Learning Outcomes

- By the end of this lecture you should be able to:
 - Discuss what **needs** to be considered when drawing up a **software deployment plan**
 - Explain the differences between the **three types of deployment** method *Big Bang*, *Phased* and *Parallel*
 - Discuss the **advantages and disadvantages** of the above
 - Explain why **software maintenance is necessary**
 - Discuss the different types of software **maintenance**
 - Explain why the **costs of maintenance** can be very high





Software Deployment

Software Deployment

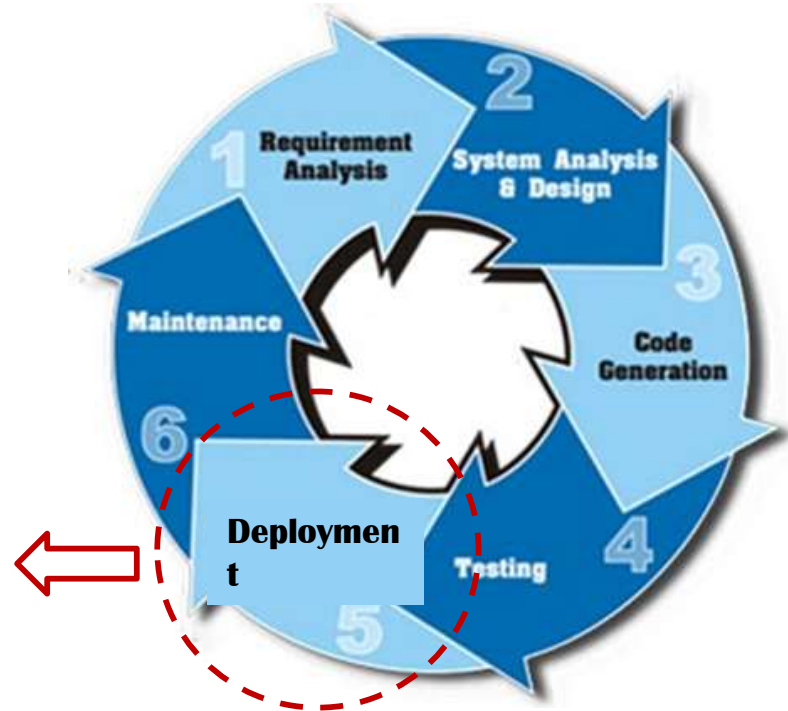
- So you've built and tested the new software system for the customer ...
- Deployment is ???
- The process of putting the finished application in the users' hands. Also known as *implementation*, *installation*, and ...
- *release into the wild*



Who is responsible for deployment?

- In the case of bespoke business systems
- Deployment is often the **responsibility of the customer**, but the developer may have provided or assisted them in creating deployment plans. It depends what kind of contract is in place.

- If a system has been developed for general release to the public e.g. a new operating system
- The **developer is responsible** for creating the deployment plan



Software Deployment

- Deployment sounds quite simple, but how do you actually roll out a new system to users including businesses and their employees?
- You need a *Deployment Plan*, but what do you need to consider?



Two Extreme Scenarios



User adulation



User frustration and refusal to work with the new system



Reality is usually somewhere in the middle

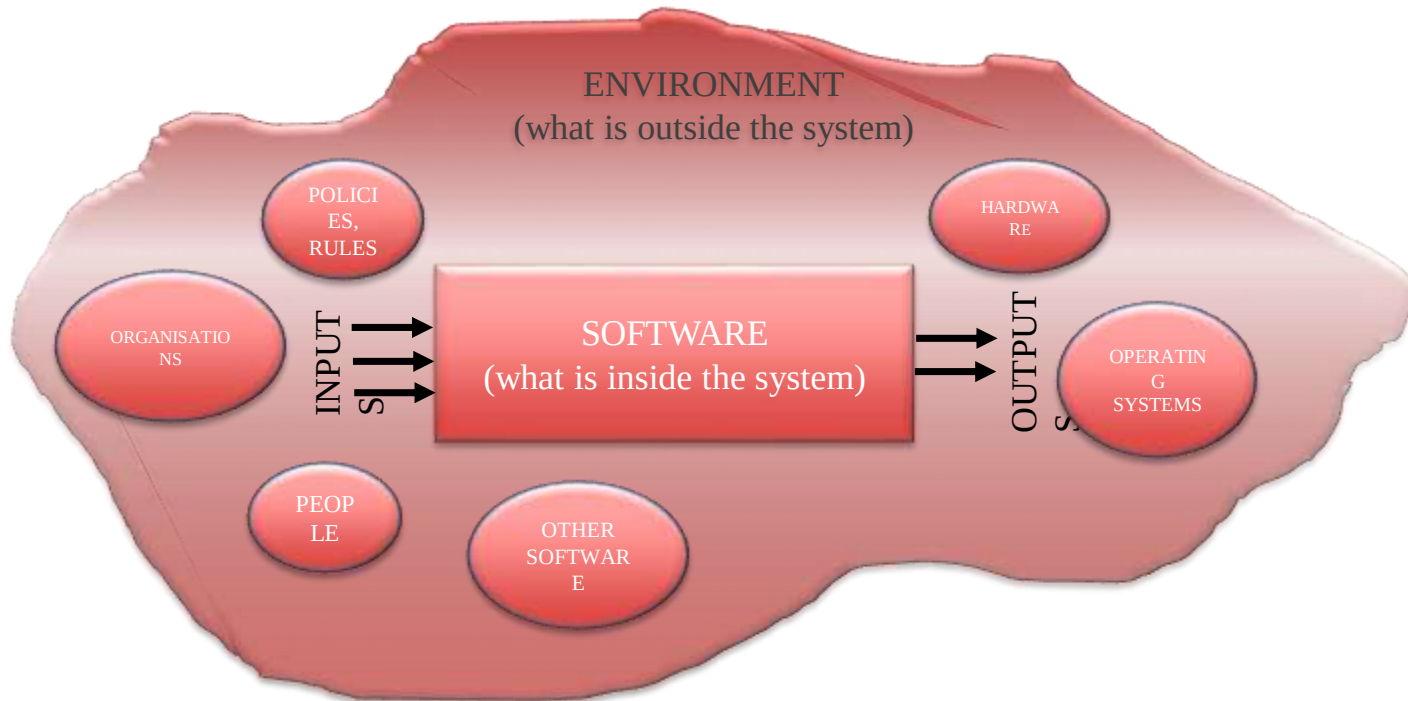
Group Exercise

- Before deploying your software system to a business that needs it, what do you have to consider?
- List at least ten items...

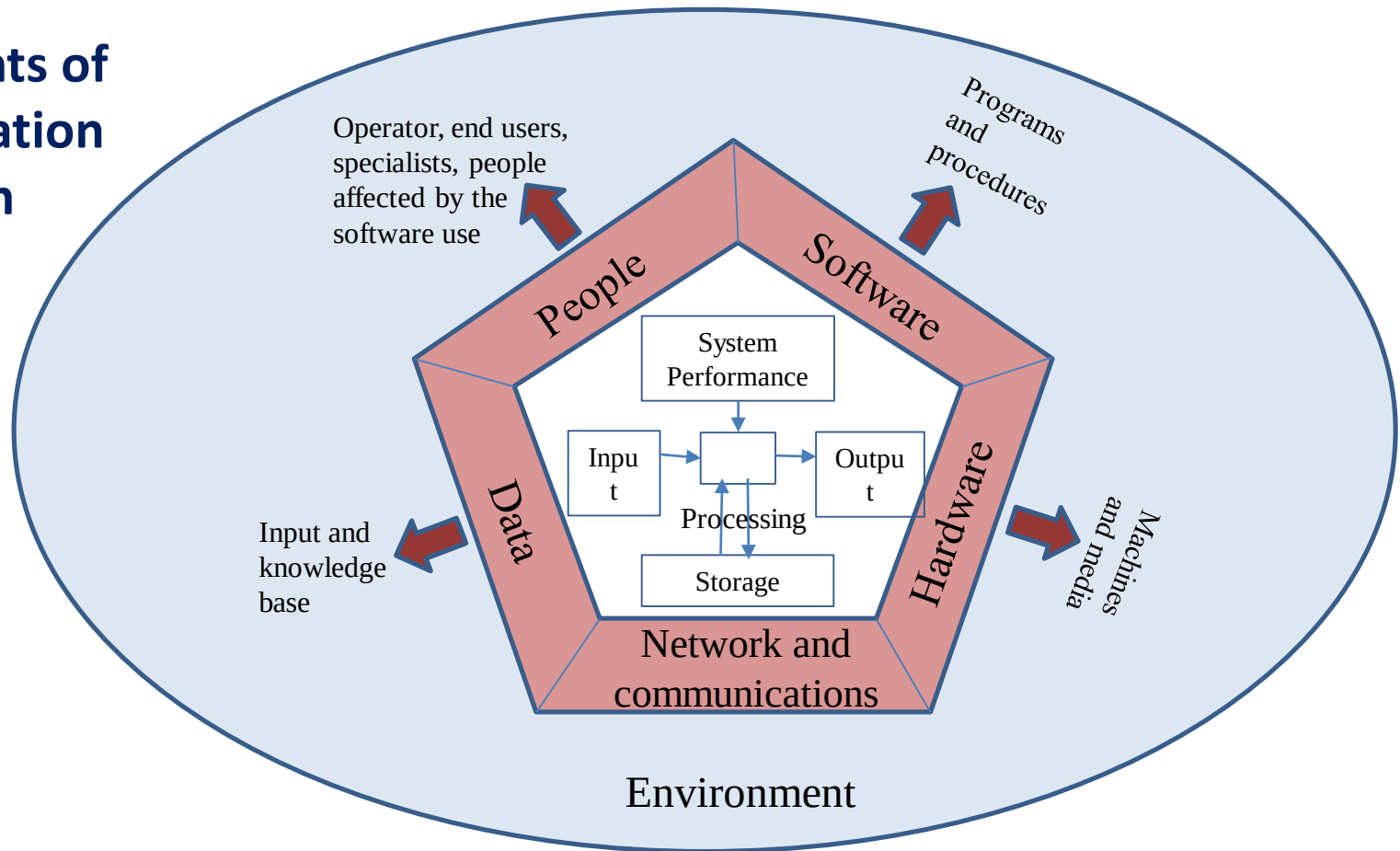
(10 mins)



Context of Software



Components of an Information System



What do we need to consider?

- Scope of the project
 - Number of users
 - Size of the application
- Environment (What is outside the system)
 - Hardware – computers, processors, cables, routers, printers, scanners, DVD burners, hard drives, networks etc.
 - Other Software (not part of the system), operating systems
 - Physical environment – cubicles, offices, desks, telephones, headsets etc.
- Existing Processes and Systems
 - What needs to change?
 - Are users ready for the change?

- Users
 - User documentation
 - Training
 - User privilege hierarchy
- Policies and rules
- Other organisations
- System inputs and outputs
- Chain of customers
- Data
 - Who is in charge of it?
 - Sensitive data
- Who takes 'ownership' of the system within the business?



Scope is Very Important!

1. Number of users. Your product could be ..

- A small tool you wrote for your own use
- A bespoke business system that a whole company will use
- An application used by millions, for example operating systems, browsers, computer games

Will a failure be inconvenience to a lot of users?

Is there a chain of use?

Will users be unable to help their customers if something goes wrong with the new system?

2. Size of the application

- How much data is involved?
- How many external systems are affected?
- Sheer quantity of code in the application

Generally, **the larger the scope the more opportunity for deployment mistakes.**

Large projects have more pieces that can go wrong and provide more combinations of little things that can add up to big problems.

Requirements

- Most of these considerations will have been addressed by requirements analysis, but we still need a **plan** for deployment that takes everything into account
- We need to think about how the introduction of a new software system affects an organisation
- Let us look at a case study that demonstrates a simple example of how deployment can go wrong



Case Study

Case Study

- Doug has responsibility for submitting funding bids for an animal charity called Paws On.
- He keeps records of all his bids (including funder name, sum asked for, project details etc.) in his own spreadsheet



Doug



Existing Processes and Systems

- Six months ago the Chief Executive of Paws On commissioned a new bespoke database system called Pawfect to manage information on funding bids, successful projects, and organisation finances and budgets
 - Doug was informed about the coming change and was told that when the new system came online he would need to input his information into Pawfect rather than his own spreadsheet
- Two weeks after the new system came online Doug's line manager identified that he had not input any of his information into Pawfect. In fact, he was still using his own spreadsheet to record it.



Group Exercise

- What was not considered thoroughly enough?

(5 mins)



Considerations

- Scope of the project
 - Number of users
 - Size of the application
- Environment (What is outside the system)
 - Hardware – computers, processors, cables, routers, printers, scanners, DVD burners, hard drives, networks etc.
 - Other Software (not part of the system), operating systems
 - Physical environment – cubicles, offices, desks, telephones, headsets etc.
- Existing Processes and Systems
 - What needs to change?
 - Are users ready for the change?

- Users
 - User documentation
 - Training
 - User privilege hierarchy
- Policies and rules
- Other organisations
- System inputs and outputs
- Chain of customers
- Data
 - Who is in charge of it?
 - Sensitive data
- Who takes 'ownership' of the system within the business?



What was not considered?

- Scope of the project
 - Number of users
 - Size of the application
- Environment (What is outside the system)
 - Hardware – computers, processors, cables, routers, printers, scanners, DVD burners, hard drives, networks etc.
 - Other Software (not part of the system), operating systems
 - Physical environment – cubicles, offices, desks, telephones, headsets etc.
- Existing Processes and Systems
 - What needs to change?
 - Are users ready for the change?

- Users
 - User documentation
 - Training
 - User privilege hierarchy
- Policies and rules
- Other organisations
- System inputs and outputs
- Chain of customers
- Data
 - Who is in charge of it?
 - Sensitive data
- Who takes 'ownership' of the system within the business?

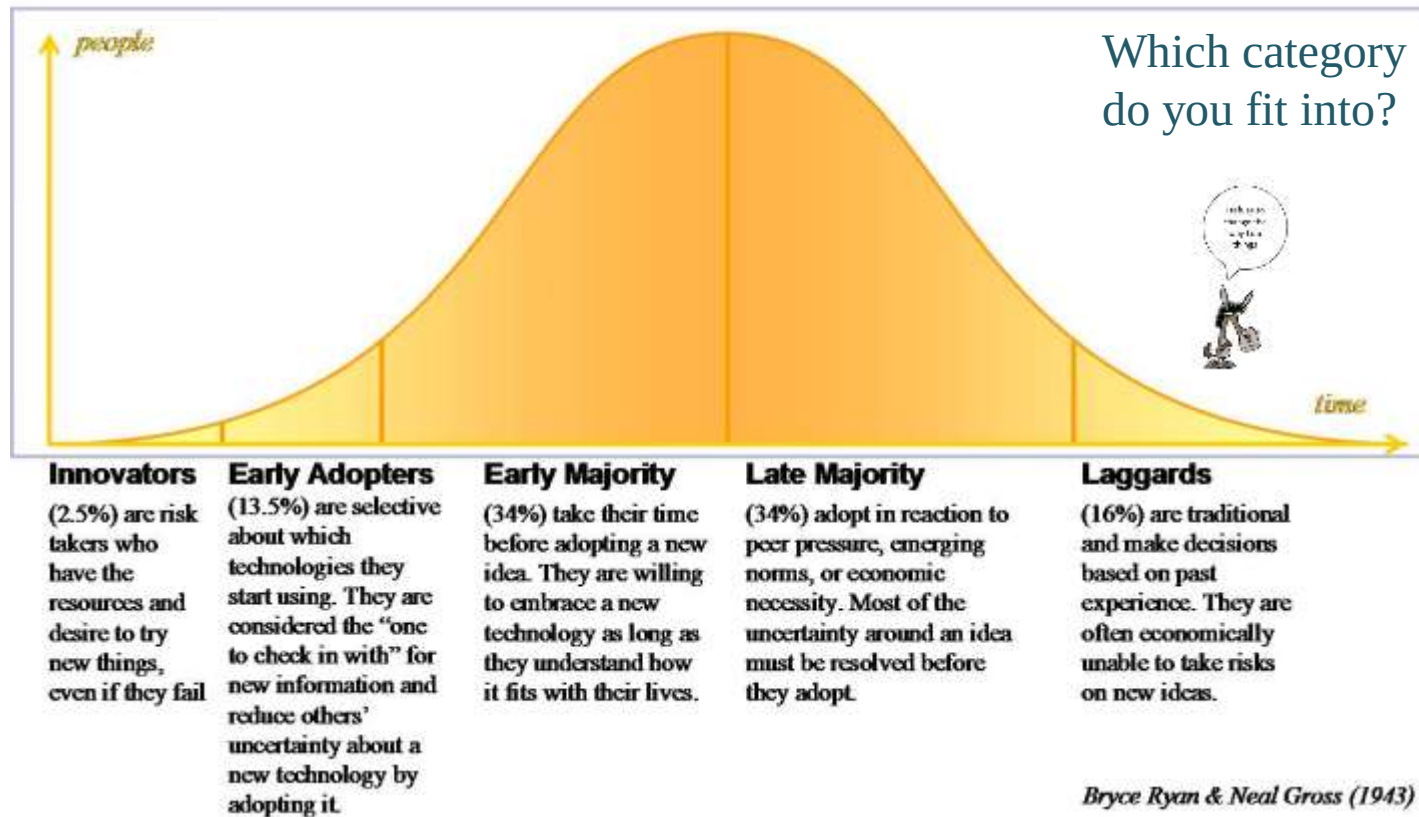
Existing Processes and Systems

- Employees can sometimes be very set in their ways and will **resist change** to the processes and systems that they have become familiar with
- This is where the **deployment plan** comes in
- There are **different models** for deploying new software systems
- **Making the plan involves selecting the best deployment model** and integrating it with appropriate training and user documentation

I refuse to
change the
way I do
things



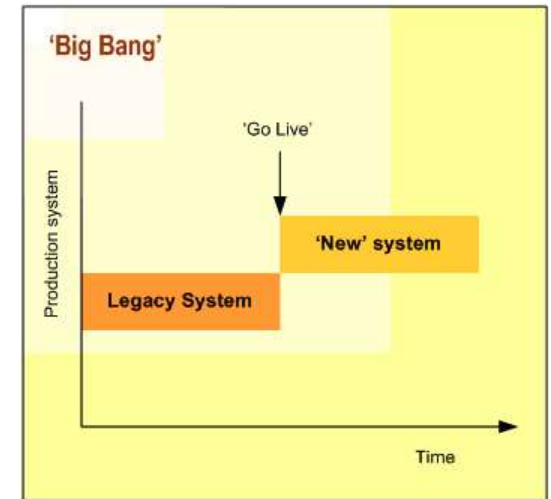
Technology Adoption Curve



Deployment Models

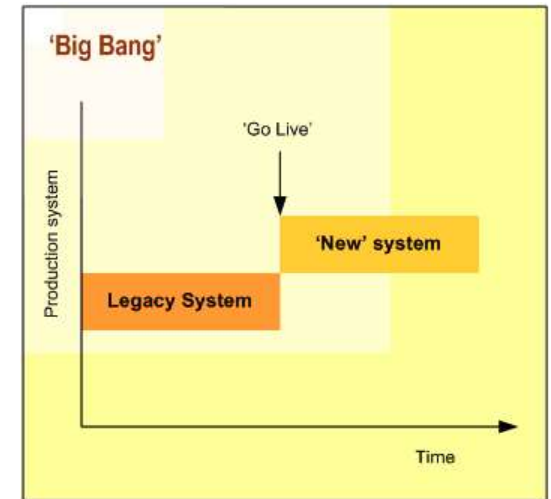
Big Bang Deployment

- A wholly functioning new system is introduced on an agreed date
- Everybody associated with the new system moves to it on that date
- The old system is not used anymore from that moment on
- Suitable for small systems



Big Bang Deployment

- **Advantages**
 - Available immediately
 - Only one system to maintain
- **Disadvantages**
 - The new systems often fail as the users are not prepared
 - Riskier than other deployment models as there are fewer learning opportunities incorporated in the approach, so much preparation is needed
 - There is an initial dip in employee performance

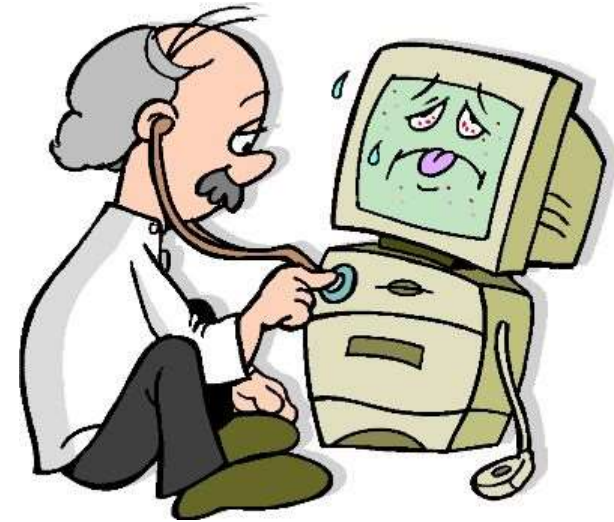


Big Bang Deployment

- Not really suitable for larger more complex systems because ...
- The sheer scale of a typical larger system means there will always be **problems that are not discovered until deployment** or even until the application has been running for a while
- The more parts of a system that there are, the greater the chance that more than one defect will surface at the same time
- A gradual deployment makes it possible to discover and handle them one by one.

Big Bang Deployment – Other Problems

- Complexity
 - A big-bang deployment requires coordination of many people and many “moving parts” that depend on each other, providing much opportunity for mistakes
- Time cost
 - Such a deployment requires lot of time (typically more than planned/expected) and there can thus be a lot of downtime when users cannot use the system.



Big Bang Deployment – Other Problems

- Difficulty in troubleshooting
 - It can be difficult to pinpoint the source of defects as the inter-dependent parts changed at the same time, maybe also changing the infrastructure
 - There can also be “panic fixes” that either cause more problems than they remove or are not good enough



Big Bang Deployment – Other Problems

- Rollback is difficult
 - Rollback is likely either impossible or equally as time-consuming and risky as the deployment itself, thus increasing the impact of defects
- Scale of Impact
 - Deploying everything to all users at the same time means that everybody will be impacted by any potential defect/error/mistake.

Phased Deployment

- The new system is installed slowly over a period of time.
- For example, users can be introduced to it in groups over time, e.g. Dept. A, Dept. B, Dept. C etc.
- Advantages
 - Spreads the cost of deployment
 - It is less disruptive than the big bang approach
 - Users can be trained over a longer period of time
- Disadvantages
 - Running two systems in parallel can cause confusion
 - The benefits of the new system can take longer to be realized

Phased Deployment

Old System

New System

Department A	Department A	Department A	Department A	Department A
Department B	Department B	Department B	Department B	Department B
Department C	Department C	Department C	Department C	Department C
Department D	Department D	Department D	Department D	Department D



Time

- In a **supermarket, the checkout system** is being upgraded to a newer version.
- However, to start with, **only one checkout counter** is changed over to the new system, while the other counters carry on with the old system.
- If the **new system does not work properly**, it would not matter so much, because only a small portion of the supermarket has been affected.
- If **it does work, staff can take turns working** on the counter with the new system to get some experience of it



Phased Deployment Example

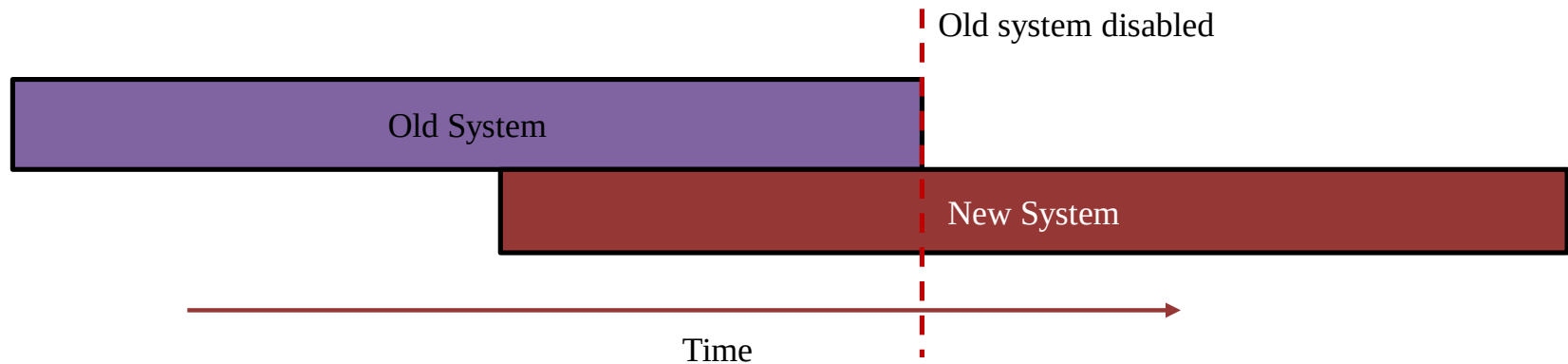
Phased Deployment Example

- After the **trial checkout counter is working perfectly**, another two might be next, then another two
- Eventually **all the counters in the supermarket system are phased in** and the new system is running everywhere
- It can take a long time until the changeover is completed, with two systems running in parallel.
- However, the **supermarket is never in danger of having to close and the staff are all able to get plenty of training** in operating the new system, so it is a much friendlier method.



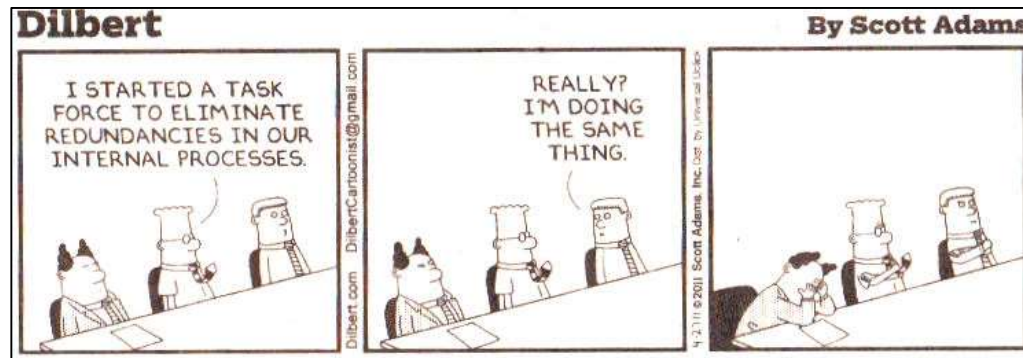
Parallel Deployment

- The old and new systems run together for a time so all the users can get used to the new system whilst still carrying out their work using the old system
- If the criteria for the new system are met, the old system is disabled.
- The process requires careful planning and control and a significant investment in time



Parallel Deployment

- During the transition, users work with both systems as they gradually learn the new software.
- There is generally some duplication of effort as, for example, data must be entered into both systems.
- The duplication can lead to data quality issues.



Deployment Method Summary

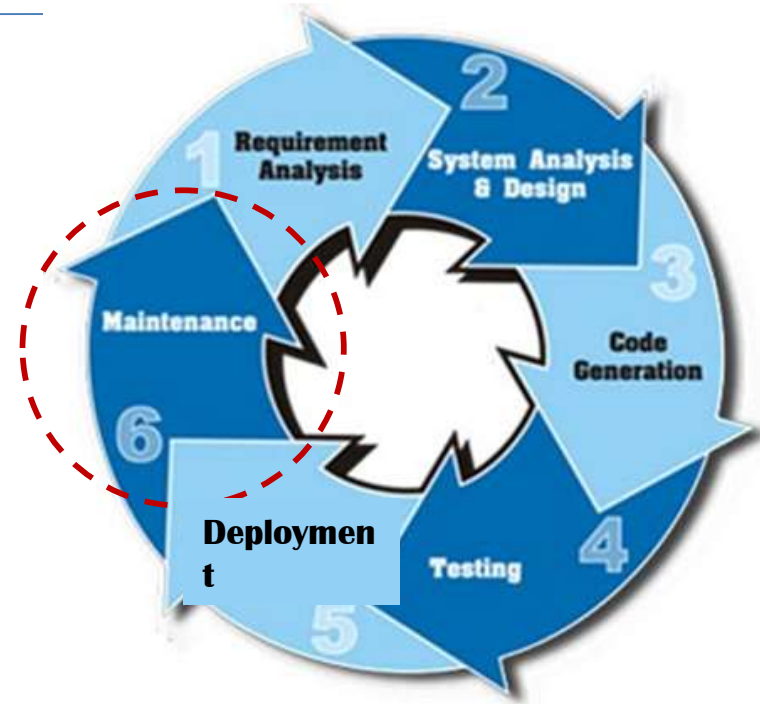
- **Big bang** - Implementation happens in a single instance. All users move to the new system on a given date.
- **Phased rollout** - Changeover occurs in phases over an extended period of time. Users move over to the new system in a series of steps.
- **Parallel adoption** - Both the legacy and new system run at the same time. Users learn the new system while working on the old. On a given date the old system is removed.

Exercise

- Returning to the Pawfect example, discuss which deployment methodology you think would have been best in helping Doug to adopt the new system?
- Agree on a system and present a case for it

(10 mins)





Software Maintenance

Group Exercise

- What reasons might a company have for modifying a software system?
- Make a list of at least three items

(5 mins)



Software Maintenance

- The IEEE Standard for Software Maintenance (STD 1219-1993) defines maintenance as:

*“Modification of a software product **after delivery** to:*

- correct faults*
- improve performance or other attributes*
- adapt the product to a modified environment”*



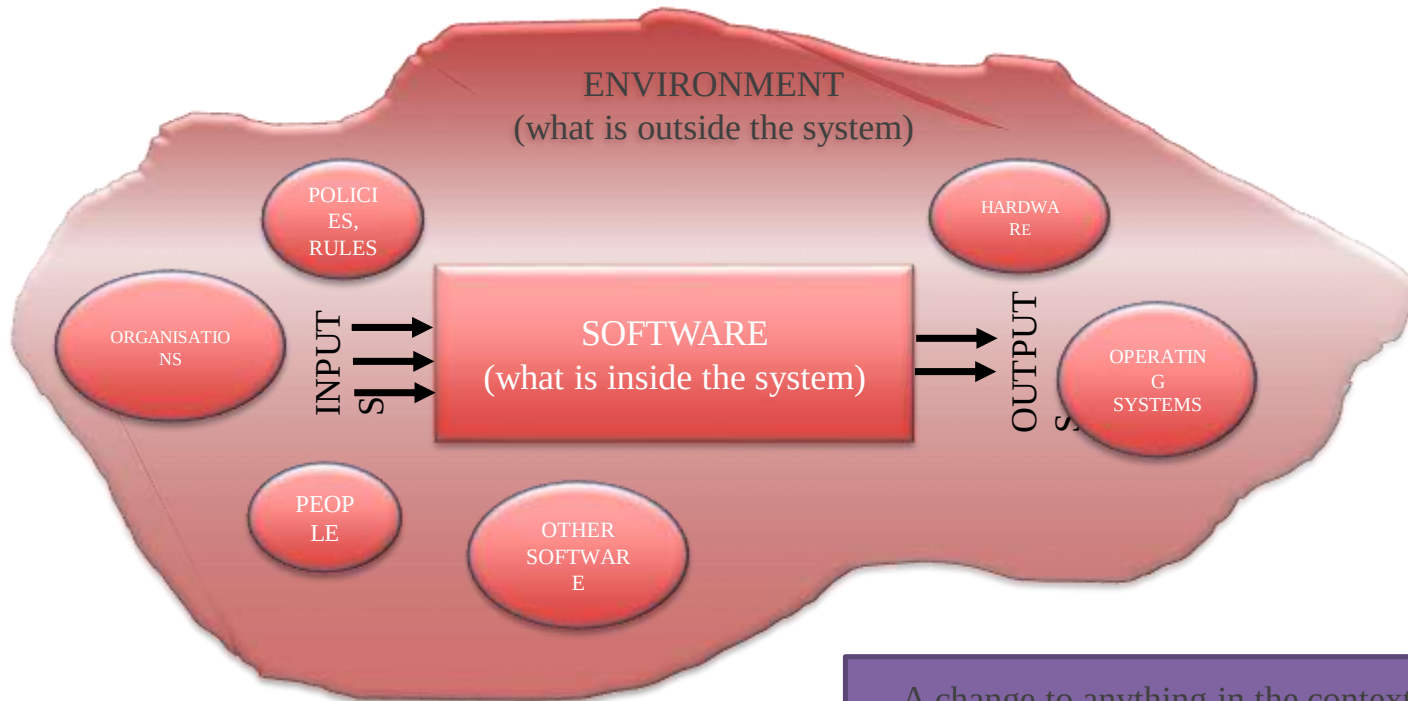
Why Modify?

1. **Company policies** or business processes might change. This might be in response to new laws, government policies, stakeholder policies etc.
2. **Bugs** might be discovered that need logging and fixing
3. The customer may ask for **new features or functions** in the software in response to feedback from the users / employees or perhaps to give a competitive advantage

Why Modify?

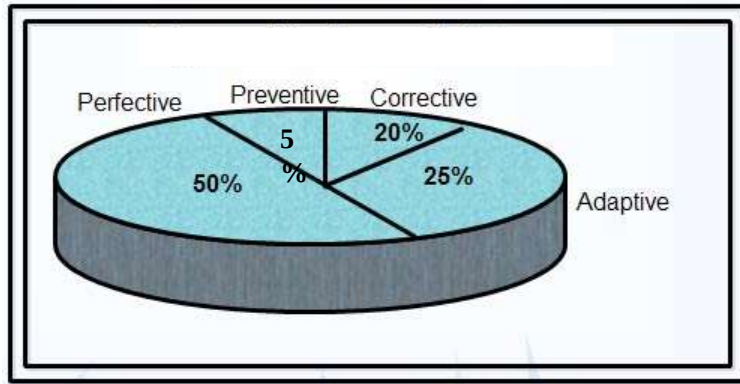
4. The **environment** e.g. the hardware / platform (such as operating system) that the customer uses may change. Perhaps software that the system depends on has been upgraded
5. **Organization changes** may prompt a need for a change to the system e.g. downsizing the company, acquiring another company, a new business direction, new branches of the company, acquiring new assets, an office move etc.

Context of Software



A change to anything in the context can mean that the software needs changing in some way

Types of Software Maintenance



Types of maintenance and percentage of time consumed for typical large project

Source:

ecomputernotes.com/software-engineering/types-of-software-maintenance

by Dinesh Thakur

- **Corrective** maintenance - concerned with fixing errors / bugs that are observed when the software is in use.
- **Adaptive** maintenance - concerned with changing the software to make it up-to-date and adaptable to a new environment e.g. a new operating system.
- **Perfective** maintenance - concerned with adding new functionalities and improving existing ones in the software.
- **Preventive** maintenance – Restructuring the code to make it more maintainable

Group Exercise

- Associate one of the types of software maintenance with the five items in the list we looked at

(5 mins)



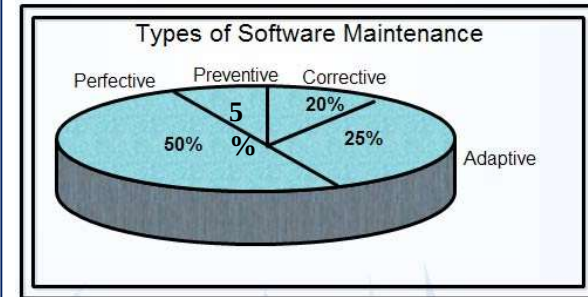
Why Modify?

1. Company policies or business processes might change. This might be in response to new laws, government policies, stakeholder policies etc. **ADAPTIVE**
2. Bugs might be discovered that need logging and fixing **CORRECTIVE**
3. The customer may ask for new features or functions in the software in response to feedback from the users / employees or perhaps to give a competitive advantage **PERFECTIVE**
4. The environment e.g. the hardware / platform (such as operating system) that the customer uses may change. Perhaps software that the system depends on has been upgraded **ADAPTIVE**
5. Organization changes may prompt a need for a change to the system e.g. downsizing the company, acquiring another company, a new business direction, new branches of the company, acquiring new assets, an office move etc. **ADAPTIVE**

ADAPTIVE? CORRECTIVE? PERFECTIVE? PREVENTIVE?

Preventive Maintenance

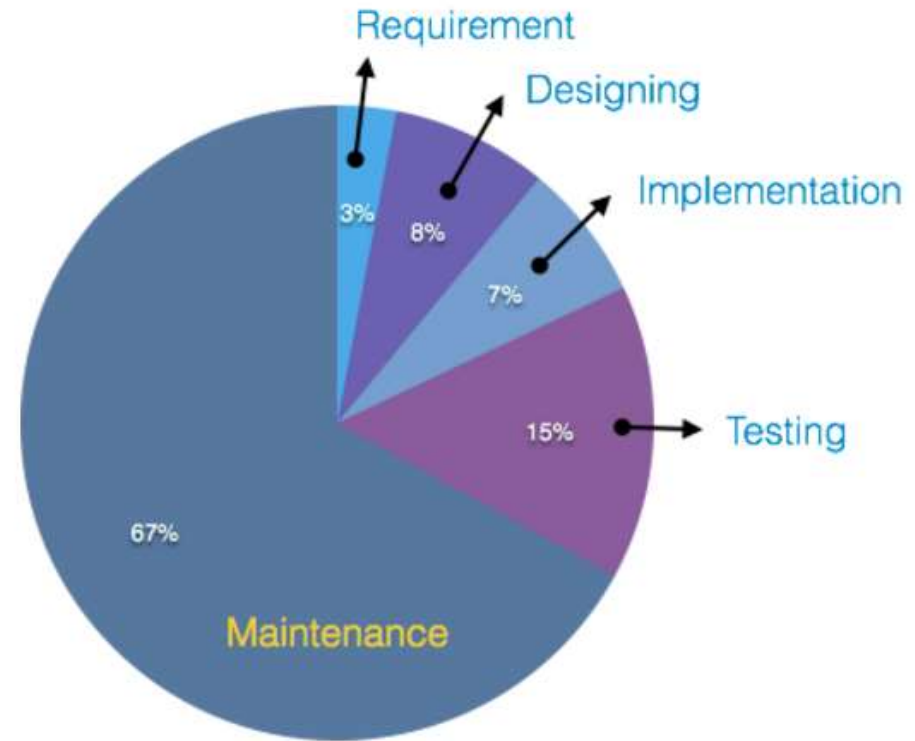
- About preventing the occurrence of errors.
- Aim is usually to reduce the software complexity and make the program easier to understand, thus increasing maintainability
- Involves code optimization, refactoring, code restructuring, and updating user documentation
- Code optimization aims to produce a faster execution time or more efficient use of storage space for example.
- Preventive maintenance accounts for only about 5% of all the maintenance activities.



Cost of Maintenance

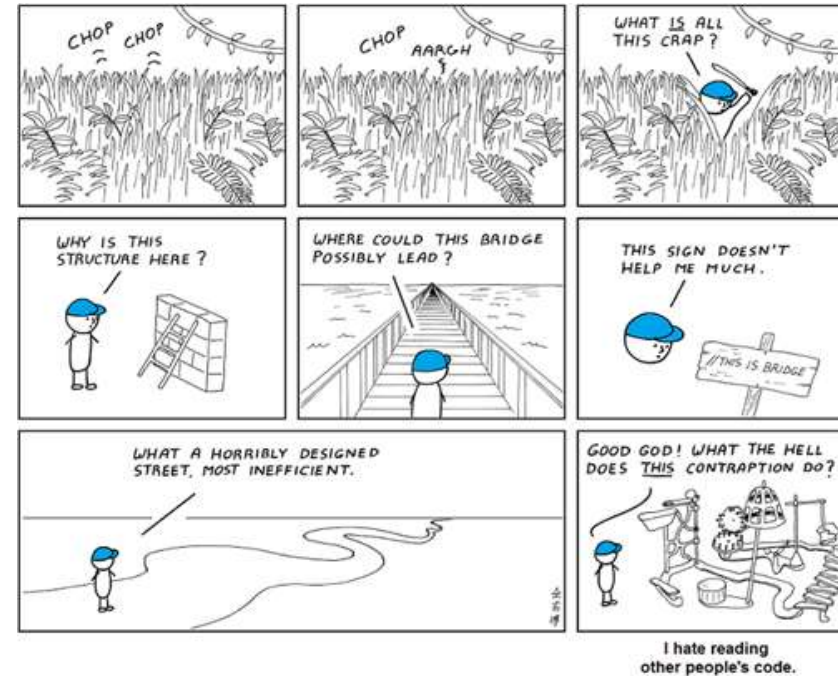
- A study found that the cost of maintenance can be as high as 67% of the cost of entire software process cycle *
- On average, the cost of software maintenance is more than 50% of all the SDLC phases.
- There are various factors that trigger maintenance costs to increase

* Source: Tutorialspoint.com



Cost of Maintenance

- Applications often live far longer than originally intended. Systems may still be in use decades after they were created
- There is a steep learning curve for developers working with someone else's code. It takes longer to work on therefore costs more.
- May be easier and quicker for them to write new code than modify someone else's old code. However, costs might still be high because the code would need more testing



Cost of Maintenance continued...

- As maintenance proceeds, code interdependencies can increase - code can become more complex and bugs harder to trace and fix
- Time pressures for resolving problems means that many developers solve problems with quick fixes or “hacks” that are poorly documented or understood
- This poor code maintenance spawns more maintenance problems



KSPs

Thanks

Next Lecture:

Review Lecture